SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN

Zdravko Blažević

PAMETNI UGOVOR ZA AUKCIJU

PROJEKTNI ZADATAK

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Zdravko Blažević

PAMETNI UGOVOR ZA AUKCIJU

PROJEKTNI ZADATAK

Mentorica :

Dr. sc. socio. Snježana Križanić

Varaždin, svibanj 2026.

Sadržaj

1. Uvod 1

2. Blockchain 2
2.1. Povijest Blockchain 2
2.2. Način rada blockchaina 3
2.3. Konsenzusni mehanizam 4
2.4. Kriptografija u blockchainu 4
2.5. Pametni ugovori (Smart Contracts) 5
2.6. Blockchain u digitalnom poslovanju 5
2.7. Prednosti i nedostaci blockchaina 6
2.7.1. Prednosti 6
2.7.2. Nedostaci 6
2.8. Budućnost blockchain tehnologije 6

3. Opis use case-a: Aukcijski pametni ugovor 8

4. Implementacija pametnog ugovora za aukciju 10
4.1. Pristup 1: Jednostavna aukcija s direktnim povratom prethodne ponude 10
4.1.1. Objašnjenje programskog rješenja 12
4.1.1.1. Struktura pametnog ugovora 12
4.1.1.2. Funkcija ponudi() 13
4.1.1.3. Funkcija zavrsiAukciju() 13
4.1.1.4. Funkcija vrijemePreostalo() 14
4.1.2. Prikaz rješenja 14
4.2. Pristup 2: Aukcija s mapiranjem za povrat sredstava 16
4.2.1. Objašnjenje programskog rješenja 19
4.2.1.1. Struktura pametnog ugovora 19
4.2.1.2. Funkcija ponudi() 20
4.2.1.3. Funkcija povuciPovrat() 20
4.2.1.4. Funkcija zavrsiAukciju() 21
4.2.1.5. Funkcija vrijemePreostalo() i mojPovrat() 21
4.2.2. Prikaz rješenja 22

5. Zaključak 27

Popis literature 28

Popis slika 29

1. Uvod

Suvremeno digitalno poslovanje sve se više oslanja na tehnologije koje omogućuju sigurnu razmjenu podataka i smanjenje potrebe za posrednicima. Takva vrsta tehnologije je blockchain koja je obrađena u ovom radu. Blockchain se može prikazati kao baza podataka o transakcijama koja se ažurira i dijeli između više računala u mreži, pri čemu se novi skup transakcija zapisuje u blok.[1]

Razvoj blockchain tehnologije najviše se povezuje s Bitcoinom, koji je osnovao peer-to-peer sustav elektroničkog novca. Glavna ideja takvog sustava je omogućiti izravnu razmjenu vrijednosti između korisnika bez potrebe za financijskom institucijom kao posrednikom. Bitcoin je također riješio problem dvostruke potrošnje pomoću mreže koja vremenski označava transakcije i povezuje ih u lanac blokova. [2] Nakon Bitcoina razvijaju se naprednije blockchain platforme, među kojima je najpoznatiji Ethereum. Ethereum ne služi samo za prijenos vrijednosti, nego omogućuje i izvršavanje programskog koda kroz pametne ugovore. Pametni ugovori automatski izvršavaju unaprijed definirana pravila, zbog čega su prikladni za automatizaciju poslovnih procesa i izradu decentraliziranih aplikacija. [3]

U samim projektu obrađuje se tema pametnog ugovora za aukciju. Kojem je cilj prikazati kako se klasičan proces aukcije može prenijeti u blockchain okruženje. U praktičnom dijelu izrađena su dva Solidity pametna ugovora. Prvi pristup prikazuje jednostavnu aukciju s direktnim povratom prethodne ponude, dok drugi pristup koristi mapiranje za spremanje povrata sredstava ponuditeljima čije su ponude nadmašene.

Za izradu praktičnog dijela korišten je jezik Solidity i Remix VM okruženje. Solidity je namijenjen izradi pametnih ugovora za Ethereum Virtualnu Mašinu, a među mogućim primjerima primjene nalaze se i aukcije.

Rad je podijeljen na teorijski i praktični dio. U teorijskom dijelu objašnjavaju se osnovni pojmovi blockchain tehnologije, način rada blockchaina, konsenzusni mehanizmi, kriptografija, pametni ugovori te primjena blockchaina u digitalnom poslovanju. U praktičnom dijelu prikazana je implementacija aukcijskog pametnog ugovora, opis use case-a, use case dijagram, programski kod i testiranje rješenja u Remix Ethereum okruženju.

2. Blockchain

Blockchain tehnologija je jedna od najbitnijih tehnologija u razvoju digitalnih inovacija i digitalnog poslovanja. Blockchain se ubraja među tehnologije koje su trenutno „u usponu", zajedno s robotikom i umjetnom inteligencijom . To pokazuje velik i potencijal za budući razvoj gospodarstva i društva.[1]

Osnovna vrijednost blockchain tehnologije je u njegovoj sposobnosti decentraliziranog upravljanja podacima. Umjesto centraliziranih sustava, gdje jedna institucija kontrolira podatke, blockchain omogućuje mreži korisnika da zajednički održavaju i verificiraju zapise. Na taj se način povećava transparentnost i smanjuju se potrebe za posrednicima. Ovakav je važan u digitalnom poslovanju jer omogućuje bolju i sigurniju razmjenu podataka i vrijednosti između različitih sudionika bez potrebe za povjerenjem u treću stranu.[2]

Jedan od problema koji blockchain rješava je problem dvostruke potrošnje (double-spending), to jest mogućnost da se ista digitalna vrijednost potroši više puta. Rješenje se bazira na mreži koja bilježi transakcije u obliku vremenski označenih blokova povezanih u lanac, pri čemu se svaka promjena mora potvrditi kroz konsenzus mreže . Na taj način osigurava se integritet i nepromjenjivost podataka.[2]

Kroz razvoj blockchain tehnologije omogućen je nastanak naprednijih sustava poput pametnih ugovora (smart contracts), koji automatski izvršavaju unaprijed definirana pravila bez potrebe za posrednicima, na ovakav način sustavi omogućuju automatizaciju poslovnih procesa, te se samim timdodatno unapređuje digitalno poslovanje.[3]

Osim u financijskom sektoru, blockchain ima primjenu i u različitim područjima digitalnog poslovanja, uključujući upravljanje digitalnom imovinom, identitetima i podacima. Njegove glavne prednosti su decentralizacija, sigurnost, transparentnost i nepromjenjivost podataka, što ga čini ključnom tehnologijom za razvoj novih digitalnih poslovnih modela .[4]

2.1. Povijest Blockchain

Razvoj blockchain tehnologije započeo je 2008. gdje su predstavljeni prvi funkcionalni model decentraliziranog sustava elektroničkog novca.Blockchain se koristi kao temelj za sustav koji omogućuje izravne transakcije između korisnika bez potrebe za centralnim autoritetom. Glavni problem koji je time riješen je problem dvostruke potrošnje (double-spending), odnosno mogućnost da se ista digitalna vrijednost potroši više puta. [2]

Bitcoin je prva generacija blockchain tehnologije, čija je primarna svrha decentralizirano bilježenje transakcija. Transakcije se grupiraju u blokove koji su međusobno povezani kriptografskim hash funkcijama, te se na taj način osigurava sigurnost i nepromjenjivost podataka. Na ovaj način omogućeno je stvaranje globalne decentralizirane knjige zapisa koja je otporna na manipulacije.[4]

Nakon Bitcoina razvijaju se napredniji blockchain sustavi, među kojima je najznačajniji Ethereum. Ethereum proširuje ideju blockchaina jer uvodi koncep opće namjene, odnosno

omogućuje izvršavanje programskog koda unutar same mreže. Prema Ethereum specifikaciji, blockchain se može promatrati kao transakcijski sustav koji kontinuirano mijenja stanje sustava kroz izvršavanje transakcija, a samim time blockchain više nije samo knjiga transakcija, već postaje platforma za razvoj različitih aplikacija.[5]

Najvažnijih Ethereumova inovacija je uvođenje pametnih ugovora (smart contracts). Pametni ugovori su programi koji se automatski izvršavaju kada su ispunjeni određeni uvjeti, čime se omogućuje automatizacija poslovnih procesa bez potrebe za posrednicima .[3]

Daljnji razvoj blockchain tehnologije išao je ka proširenju primjene izvan financijskog sektora. Danas se blockchain koristi u različitim područjima poput digitalnog poslovanja, upravljanja podacima, logistike i digitalnih identiteta. Prema raznim istraživanjima, blockchain se sve više integrira i u napredna digitalna okruženja poput metaverzuma, gdje donose sigurnu pohranu podataka, upravljanje digitalnom imovinom i interoperabilnost sustava.[4]

Zaključno, razvoj blockchain tehnologije može se podijeliti u nekoliko faza: početak s Bitcoinom kao decentraliziranim sustavom plaćanja, razvoj Ethereuma kao platforme za izvršavanje pametnih ugovora te suvremenu fazu u kojoj blockchain postaje temelj za razvoj kompleksnih digitalnih sustava i ekosustava.

2.2. Način rada blockchaina

Osnova blockchain tehnologija je peer-to-peer (P2P) mreža u kojoj svi sudionici (čvorovi) međusobno komuniciraju bez centralnog posrednika. U takvom sustavu svaki čvor ima pristup istoj verziji knjige zapisa te sudjeluje u provjeri i pohrani podataka.[2]

Proces rada blockchaina započinje tako što se kreiraju transakcije. Transakcije su digitalni potpisani instrukcija kojom se prenosi vrijednost ili podaci između korisnika. Svaka transakcija mora biti verificirana od strane mreže prije nego što bude trajno zapisana u blockchain. U validaciju transakcije uključuje se provjeru autentičnosti, to jest digitalnog potpisa, kao i provjeru raspoloživih sredstava ili valjanosti podataka .[5] Ṅakon validacije, transakcije se grupiraju u blokove. Blok je skup potvrđenih transakcija zajedno s dodatnim informacijama poput hash vrijednosti prethodnog bloka i vremenske oznake (timestamp). Upravo povezivanje blokova putem hash funkcija omogućuje stvaranje lanca blokova (blockchain) i samim time se osigurava nepromjenjivost podataka, jer svaka promjena u jednom bloku zahtijeva promjenu svih sljedećih blokova. [5]

Dodavanje novih blokova u blockchain odvija se kroz proces konsenzus među čvorovima mreže. Konsenzus je mehanizam kojim sudionici mreže postižu dogovor o tome koja verzija blockchaina je ispravna. U Bitcoin sustavu koristi se Proof of Work mehanizam, gdje čvorovi (rudari) rješavaju složene matematičke probleme kako bi dodali novi blok u lanac. [2]

Važan doo rada blockchaina je mehanizam vremenskog žiga (timestamp), koji omogućuje kronološki redoslijed transakcija. Svaki blok sadrži vremensku oznaku koja potvrđuje kada je blok kreiran, čime se sprječava manipulacija redoslijedom transakcija i osigurava redosljed sustava.[4]

Blockchain funkcionira kao distribuirani sustav u kojem se stanje mreže ažurira kroz izvršavanje transakcija. Blockchain se može promatrati kao sustav promjene stanja, gdje svaka nova transakcija mijenja trenutno stanje sustava u novo stanje.

2.3. Konsenzusni mehanizam

Konsenzus mehanizmi daje mogućnost svim čvorovima u blockchain mreži da postignu dogovor o valjanosti transakcija i stanju sustava. U decentraliziranom sustavu, gdje ne postoji centralni autoritet, konsenzus je nužan kako bi postojala sigurna jedinstvena i pouzdana verzija podataka te spriječile zlouporabe poput dvostruke potrošnje.[2]

Osnovna ideja konsenzusa je da mreža sudionika zajednički potvrđuje transakcije i odlučuje koji će blok biti dodan u blockchain. samim tim se osigurava sinkronizacija podataka među svim čvorovima i stabilnost sustava. Bez konsenzusa bi različiti sudionici mogli imati različite verzije blockchaina, što bi dovelo do nepouzdanosti sustava.[4] Najpoznatiji konsenzus mehanizam je Proof of Work (PoW), korišten u Bitcoinu, gdje čvorovi (rudari) rješavaju složene matematičke zadatke kako bi dodali novi blok u lanac Bitcoin: A Peer-to-Peer Electronic Cash System. Ovaj pristup osigurava veliku sigurnosti jer zahtijeva značajnu računalnu snagu, ali istovremeno dovodi do velike potrošnje energije.

Zbog tih nedostataka razvijeni su alternativni mehanizmi poput Proof of Stake (PoS), koji se temelji na količini kriptovalute koju korisnik posjeduje, odnosno njegovom udjelu u mreži. Time se smanjuje potreba za velikom računalnom snagom i povećava učinkovitost sustava.[4]

2.4. Kriptografija u blockchainu

Kriptografija je temelj blockchain tehnologije jer osigurava sigurnost, integritet i pouzdanost podataka unutar sustava. Blockchain koristi različite kriptografske metode kako bi zaštitio transakcije i spriječio neovlaštene promjene podataka.

Prvi element sigurnosti su hash funkcije. Hash funkcija pretvara ulazne podatke proizvoljne veličine u jedinstvenu izlaznu vrijednost fiksne duljine. U blockchainu se hash koristi za identifikaciju blokova i povezivanje podataka. Svaki blok sadrži hash prethodnog bloka, čime se stvara lanac blokova. Promjena bilo kojeg podatka u bloku rezultira promjenom hash vrijednosti, što odmah ukazuje na pokušaj manipulacije.[4] Digitalni potpisi je isto tako kriptografski mehanizam. Svaka transakcija u blockchainu mora biti digitalno potpisana od strane pošiljatelja, te se na taj način potvrđuje autentičnost i vlasništvo nad sredstvima. Digitalni potpisi omogućuju provjeru identiteta korisnika bez otkrivanja njihovih privatnih ključeva.[5]

Sigurnost i integritet podataka dobiva se kombinacijom hash funkcija i digitalnih potpisa. Hash funkcije omogućuje da podaci nisu mijenjani, dok digitalni potpisi osigrava da su transakcije izvršene od strane ovlaštenih korisnika. Na taj način blockchain sustav postaje otporan na manipulacije i prijevare.[2]

Povezanost blokova putem hash funkcija je važna za nepromjenjivost blockchaina. Svaki

blok referencira prethodni blok kroz njegov hash, te se na taj način osigurava kontinuitet lanca. Ako bi se pokušala izmjena nekog bloka, bilo bi potrebno promijeniti sve sljedeće blokove, što je u praksi gotovo nemoguće zbog distribuirane prirode mreže.[4]

2.5. Pametni ugovori (Smart Contracts)

Pametni ugovori (engl. smart contracts) su računalni programi koji se izvršavaju unutar blockchain mreže i automatski provode unaprijed definirana pravila. Njihova osnovna funkcija je izvršavanje dogovora između sudionika bez potrebe za posrednikom, pri čemu se uvjeti ugovora zapisuju u obliku programskog koda.[3]

Jedna od karakteristika pametnih ugovora je njihovo automatsko izvršavanje. Kada su zadani uvjeti ispunjeni, ugovor se izvršava bez dodatne intervencije korisnika. Time se smanjuje mogućnost za pogrešku, povećava se brzina izvršavanja i eliminira potreba za povjerenjem između sudionika.[4] Pametni ugovori imaju široku primjenu u decentraliziranim aplikacijama (dApps). Oni omogućuju razvoj sustava za upravljanje digitalnom imovinom, glasovanje, financijske usluge i razne druge poslovne procese.[5].

Najpoznatija platforma za razvoj i izvršavanje pametnih ugovora je Ethereum. Za razliku od Bitcoina, koji je primarno namijenjen prijenosu vrijednosti, Ethereum omogućuje izvršavanje programskog koda unutar blockchaina. Prema Ethereum specifikaciji, sustav se može promatrati kao platforma koja podržava izvođenje različitih aplikacija kroz promjene stanja sustava . Time Ethereum predstavlja temelj za razvoj decentraliziranih aplikacija i naprednih digitalnih sustava.[5].

2.6. Blockchain u digitalnom poslovanju

Blockchain tehnologija je krucijalna u razvoju digitalnog poslovanja, posebno u kontekstu Industrije 4.0. Kao jedna od najvažnijih tehnologija uz umjetnu inteligenciju, velike podatke i računarstvo u oblaku, blockchain omogućuje sigurnu, transparentnu i decentraliziranu razmjenu podataka između sudionika poslovnih procesa.[1]

Prednosti blockchaina leži u povezivanju različitih sustava i organizacija. Kroz decentraliziranu strukturu omogućuje se interoperabilnost, ti jest sposobnost različitih sustava da međusobno komuniciraju i razmjenjuju podatke na siguran i pouzdan način.[4]

Blockchain ima široku primjenu u različitim područjima digitalnog poslovanja. U financijskom sektoru koristi se za transakcije bez potrebe za bankama kao posrednicima. U logistici omogućuje praćenje robe kroz cijeli opskrbni lanac, čime se smanjuje mogućnost prijevara. U e-poslovanju omogućuje sigurnu razmjenu podataka i digitalne imovine, kao i automatizaciju procesa putem pametnih ugovora .

Jedna od najvećih prednosti blockchaina je uklanjanje posrednika u poslovnim procesima. Tradicionalni sustavi često zahtijevaju treće strane za provjeru i potvrdu transakcija, što povećava troškove i vrijeme obrade. Blockchain omogućuje izravnu razmjenu između sudi-

onika, čime se smanjuju troškovi, povećava brzina transakcija i poboljšava pouzdanost sustava .[2]

2.7. Prednosti i nedostaci blockchaina

Blockchain tehnologija donosi brojne prednosti u odnosu na tradicionalne sustave, ali istovremeno ima i određene nedostatke koji utječu na njezinu širu primjenu.

2.7.1. Prednosti

Jedna od osnovnih prednosti blockchaina je decentralizacija. Sustav ne ovisi o centralnom autoritetu, već se podaci distribuiraju među svim sudionicima mreže. Time se povećava otpornost sustava i smanjuje mogućnost manipulacije podacima. Sigurnost je još jedna važnih prednost. Pomoću kriptografskih metoda, poput hash funkcija i digitalnih potpisa, blockchain osigurava visoku razinu zaštite podataka i transakcija. Svaka promjena podataka zahtijevala bi izmjenu svih povezanih blokova, što je u praksi izuzetno teško ostvarivo. [4]

Transparentnost omogućuje svim sudionicima mreže uvid u transakcije, čime se povećava povjerenje između korisnika. Svi zapisi su javno dostupni i mogu se provjeriti, što smanjuje mogućnost prijevara . Nepromjenjivost podataka znači da jednom zapisani podaci u blockchainu ne mogu biti izmijenjeni ili obrisani. [4]

2.7.2. Nedostaci

Unatoč brojnim prednostima, blockchain ima i određene nedostatke. Jedan od glavnih problema je skalabilnost. Kako broj korisnika i transakcija raste, sustav može postati sporiji i manje učinkovit, što je izazov za masovnu primjenu. Potrošnja resursa posebno je izražena kod Proof of Work mehanizma, koji zahtijeva veliku količinu računalne snage i energije. To dovodi do visokih troškova i negativnog utjecaja na okoliš.[4]

Složenost implementacije također jedan od problema. Razvoj i integracija blockchain sustava zahtijevaju posebna tehnička znanja i prilagodbu postojećih poslovnih procesa, što može otežati njegovu primjenu u praksi .

2.8. Budućnost blockchain tehnologije

Blockchain tehnologija ima veliki potencijal za daljnji razvoj i primjenu u budućim digitalnim sustavima. Posebno je važna njezina uloga u razvoju metaverzuma, gdje omogućuje sigurnu pohranu podataka, upravljanje digitalnom imovinom i provođenje transakcija bez centralnog autoriteta.[4]

Jedno od važnih područja primjene blockchaina u budućnosti je digitalni identitet. Blockchain omogućuje korisnicima da upravljaju vlastitim identitetom bez potrebe za centraliziranim institucijama, čime je povećana sigurnost i privatnost podataka. Blockchain također ima važnu

ulogu u razvoju digitalnih ekosustava. Omogućuje povezivanje različitih sustava, organizacija i korisnika u jedinstvenu mrežu u kojoj se podaci mogu sigurno razmjenjivati.[4]

U budućnosti se očekuje i sve veća integracija blockchaina s drugim naprednim tehnologijama poput umjetne inteligencije (AI) i Interneta stvari (IoT). Kombinacija blockchaina i AI omogućuje automatizaciju i analizu podataka na sigurn način, dok integracija s IoT sustavima omogućuje pouzdanu razmjenu podataka između uređaja. Ove tehnologije zajedno stvaraju temelje za razvoj pametnih sustava i digitalnih infrastruktura nove generacije.[4]

3. Opis use case-a: Aukcijski pametni ugovor

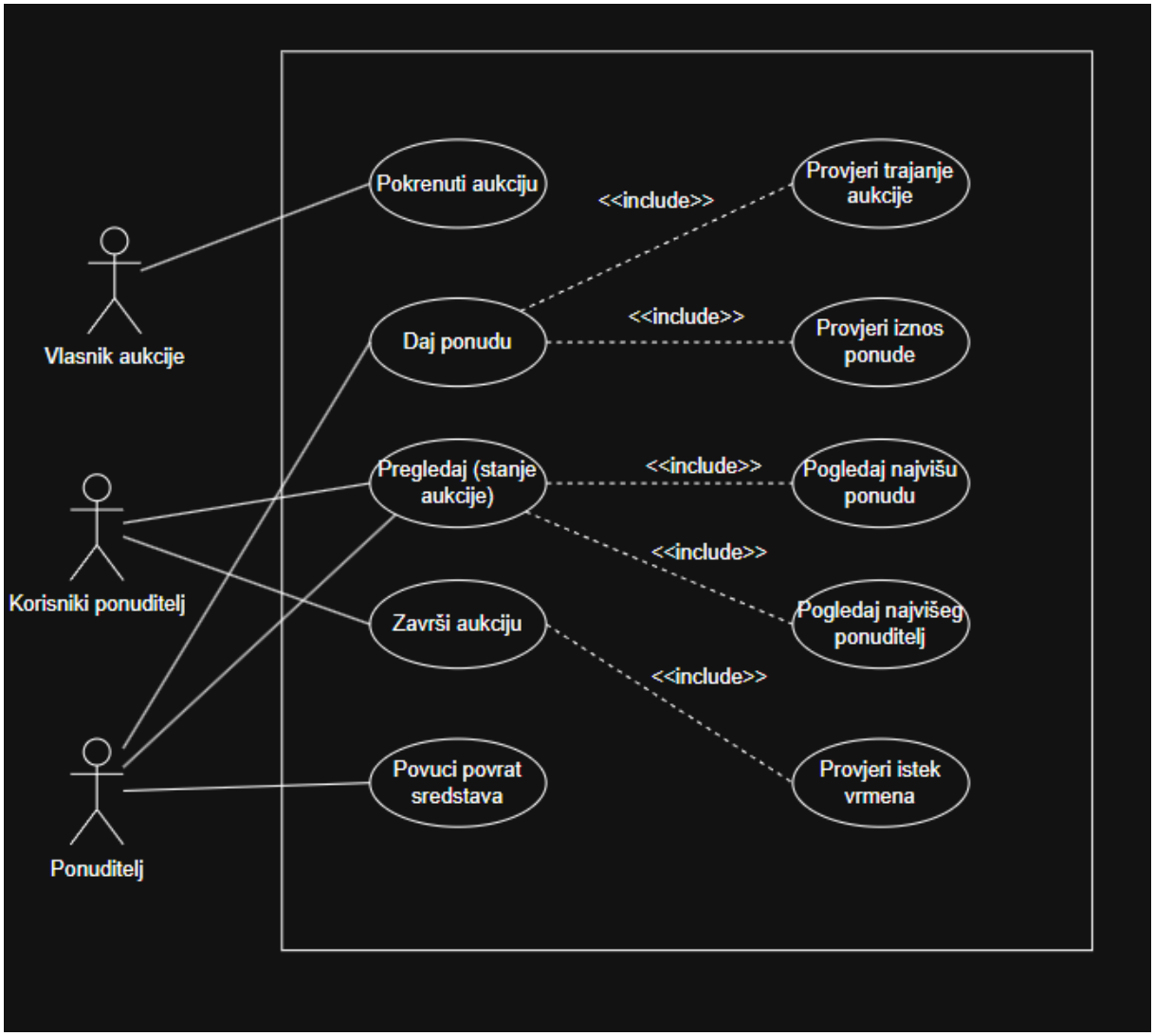
Cilj ovog use case-a je omogućiti provedbu digitalne aukcije u kojoj korisnici šalju ponude u Etheru, a nakon isteka zadanog vremena pobjednik postaje korisnik s najvećom ponudom. Pametni ugovor je prikladan za ovaj slučaj jer se izvršava transakcijama korisnika i sastoji se od funkcija i podataka koji su pohranjeni na adresi Ethereum blockchaina. Solidity je korišten zato što je namijenjen izradi pametnih ugovora za Ethereum Virtualnu Mašinu, a aukcije se ubrajaju među moguće primjene takvih ugovora.

Aukcija se pokreće tako da vlasnik deploya ugovor u Remix VM okruženju i u konstruktor unese naziv predmeta aukcije te trajanje aukcije u sekundama. Konstruktor se koristi za početno postavljanje stanja ugovora i izvršava se samo jednom, prilikom kreiranja ugovora. Tijekom trajanja aukcije ponuditelji pozivaju funkciju ponudi() i uz transakciju šalju određeni iznos Ethera. Adresa korisnika koji poziva funkciju očitava se pomoću msg.sender, poslani iznos pomoću msg.value, a vrijeme aukcije pomoću block.timestamp.

Ugovor provjerava je li aukcija još aktivna i je li nova ponuda veća od trenutne najviše ponude. Ako su uvjeti zadovoljeni, ugovor sprema novog najvišeg ponuditelja i novi najveći iznos ponude. Ako uvjeti nisu zadovoljeni, transakcija se odbija.

Nakon isteka vremena poziva se funkcija zavrsiAukciju(). Tada se aukcija označava kao završena, pobjednik ostaje zapisan u varijabli najvisiPonuditelj, a najveća ponuda u varijabli najvisaPonuda. Na slici 1. prikaz je use case dijagram.

U drugom pristupu koristi se i mapiranje mapping(address => uint) public povrati, kojim se za svaku adresu sprema iznos koji korisnik može povući ako je njegova ponuda nadmašena. Mapiranje sprema podatke u obliku para ključ-vrijednost i može povezati Ethereum adresu s pripadajućom vrijednosti.



Slika 1: Prkaz use case dijagrama (Izvor: vlastita izrada)

4. Implementacija pametnog ugovora za aukciju

U praktičnom dijelu projekta izrađena su dva pametna ugovora za provedbu digitalne aukcije. Oba ugovora imaju isti osnovni cilj, omogućiti korisnicima da tijekom određenog vremena šalju ponude u Etheru, pri čemu nakon završetka aukcije pobjeđuje korisnik koji je poslao najveću ponudu.

Prvi ugovor, AukcijaDirektanPovrat, predstavlja jednostavniji pristup. Kada novi ponuditelj pošalje veću ponudu, prethodnom najvišem ponuditelju se odmah vraća njegov iznos. Ovaj pristup je pregledan i jednostavan za testiranje jer se cijela logika ponude i povrata događa unutar funkcije ponudi().

Drugi ugovor, AukcijaMapiranjePovrata, koristi napredniji pristup. Kod njega se prethodna ponuda ne vraća odmah, nego se iznos za povrat sprema u mapiranje povrati. Mapiranje služi za pohranu podataka u obliku para ključ-vrijednost, a u ovom slučaju povezuje Ethereum adresu korisnika s iznosom koji taj korisnik može kasnije povući.

Za kontrolu vremena aukcije koristi se block.timestamp, koji predstavlja vremensku oznaku trenutnog bloka. Adresa korisnika koji poziva funkciju dobiva se pomoću msg.sender, a iznos poslan uz transakciju pomoću msg.value.

U drugom pristupu koriste se i funkcijski modifikatori samoPrijeKraja i samoNakonKraja. Oni služe za kontrolu kada se određena funkcija smije izvršiti. Funkcija ponudi() smije se izvršiti samo dok aukcija traje, dok se funkcija zavrsiAukciju() smije izvršiti tek nakon isteka vremena aukcije. Funkcijski modifikatori dodaju preduvjete funkcijama, a require provjerava istinitost uvjeta.

Oba ugovora prikazuju isti poslovni proces, ali na različit način. Prvi pristup je jednostavniji i služi za osnovno razumijevanje aukcije, dok drugi pristup bolje prikazuje korištenje mapiranja i razdvajanje logike ponude od logike povrata sredstava.

4.1. Pristup 1: Jednostavna aukcija s direktnim povratom prethodne ponude

Prvi pristup je jednostavnija verzija aukcijskog pametnog ugovora u kojoj se prethodnom najvišem ponuditelju sredstva vraćaju odmah nakon što se pojavi nova veća ponuda. Ugovor je izrađen u Solidityju i koristi osnovne elemente pametnog ugovora, statičke varijable za pohranu stanja aukcije, konstruktor za početno postavljanje vrijednosti, funkcije za izvođenje radnji te globalne varijable za dohvat podataka o korisniku, poslanom iznosu i vremenu izvršavanja.

Funkcije se koriste za podjelu programa na manje cjeline koje se mogu pozivati prema potrebi, dok se konstruktor izvršava samo jednom, prilikom kreiranja ugovora, i koristi se za inicijalizaciju početnog stanja contracta. U ovom ugovoru konstruktor sprema vlasnika aukcije, naziv predmeta i vrijeme završetka aukcije.

Glavna funkcija ovog pristupa je ponudi(). Ona prima Ether, provjerava traje li aukcija i

je li nova ponuda veća od trenutne najviše ponude. Ako je uvjet ispunjen, prethodnom najvišem ponuditelju odmah se vraća njegova ponuda, a novi korisnik postaje najviši ponuditelj. Funkcija zavrsiAukciju() završava aukciju nakon isteka vremena i prebacuje pobjednički iznos vlasniku ugovora. U nastavku je prikazan kod 1. pristupa.

```
pragma solidity >=0.7.0 <0.9.0;

contract AukcijaDirektanPovrat {
    address payable public vlasnik;
    string public predmetAukcije;
    uint public vrijemeZavrsetka;

    address payable public najvisiPonuditelj;
    uint public najvisaPonuda;

    bool public aukcijaZavrsena;

    event NovaPonuda(address ponuditelj, uint iznos);
    event AukcijaZavrsena(address pobjednik, uint iznos);

    constructor(string memory _predmetAukcije, uint _trajanjeUSekundama) {
        vlasnik = payable(msg.sender);
        predmetAukcije = _predmetAukcije;
        vrijemeZavrsetka = block.timestamp + _trajanjeUSekundama;
    }

    function ponudi() public payable {
        require(block.timestamp < vrijemeZavrsetka, "Aukcija␣je␣zavrsila.");
        require(msg.value > najvisaPonuda, "Ponuda␣mora␣biti␣veca␣od␣trenutne␣
            najvise␣ponude.");

        if (najvisaPonuda > 0) {
            najvisiPonuditelj.transfer(najvisaPonuda);
        }

        najvisiPonuditelj = payable(msg.sender);
        najvisaPonuda = msg.value;

        emit NovaPonuda(msg.sender, msg.value);
    }

    function zavrsiAukciju() public {
        require(block.timestamp >= vrijemeZavrsetka, "Aukcija␣jos␣traje.");
        require(!aukcijaZavrsena, "Aukcija␣je␣vec␣zavrsena.");

        aukcijaZavrsena = true;

        if (najvisaPonuda > 0) {
            vlasnik.transfer(najvisaPonuda);
        }

        emit AukcijaZavrsena(najvisiPonuditelj, najvisaPonuda);
    }
```

```
    function vrijemePreostalo() public view returns (uint) {
        if (block.timestamp >= vrijemeZavrsetka) {
            return 0;
        }

        return vrijemeZavrsetka - block.timestamp;
    }
}
```

4.1.1. Objašnjenje programskog rješenja

4.1.1.1. Struktura pametnog ugovora

Ovaj dio koda prikazuje osnovnu strukturu pametnog ugovora $AukcijaDirektanPovrat$. U ugovoru se definiraju glavne varijable potrebne za provedbu aukcije: vlasnik aukcije, predmet aukcije, vrijeme završetka, najviši ponuditelj, najviša ponuda i informacija o tome je li aukcija završena.

Varijabla vlasnik označena je kao address payable jer vlasnik nakon završetka aukcije prima iznos najviše ponude. Varijabla $najvisiPonuditelj$ također je označena kao address payable jer se prethodnom najvišem ponuditelju sredstva vraćaju odmah kada ga nadmaši nova veća ponuda.

Događaji $NovaPonuda$ i $AukcijaZavrsena$ koriste se za bilježenje važnih radnji u ugovoru. $NovaPonuda$ se pokreće kada korisnik uspješno pošalje novu najvišu ponudu, dok se $AukcijaZavrsena$ pokreće nakon završetka aukcije.

Konstruktor se izvršava samo jednom, prilikom kreiranja ugovora. U njemu se postavlja vlasnik aukcije pomoću msg.sender, sprema se naziv predmeta aukcije i računa se vrijeme završetka pomoću block.timestamp i trajanja aukcije koje korisnik unese prilikom deployanja ugovora.

```
contract AukcijaDirektanPovrat {
    address payable public vlasnik;
    string public predmetAukcije;
    uint public vrijemeZavrsetka;

    address payable public najvisiPonuditelj;
    uint public najvisaPonuda;

    bool public aukcijaZavrsena;

    event NovaPonuda(address ponuditelj, uint iznos);
    event AukcijaZavrsena(address pobjednik, uint iznos);

    constructor(string memory _predmetAukcije, uint _trajanjeUSekundama) {
        vlasnik = payable(msg.sender);
        predmetAukcije = _predmetAukcije;
        vrijemeZavrsetka = block.timestamp + _trajanjeUSekundama;
```

```
    }
```

4.1.1.2. Funkcija ponudi()

Funkcija $ponudi()$ služi za slanje nove ponude u prvom pristupu aukcije. Funkcija je označena kao $payable$, što znači da korisnik prilikom poziva funkcije može poslati određeni iznos Ethera.

Na početku funkcije provjerava se traje li aukcija pomoću block.timestamp. Ako je vrijeme aukcije isteklo, funkcija se neće izvršiti. Nakon toga se pomoću $require$ provjerava je li poslani iznos veći od trenutne najviše ponude. Poslani iznos dohvaća se pomoću msg.value. Ako već postoji prethodna najviša ponuda, ona se odmah vraća prethodnom najvišem ponuditelju pomoću $transfer$. Nakon toga se adresa novog ponuditelja sprema u varijablu $najvisiPonuditelj$, a poslani iznos u varijablu $najvisaPonuda$.

Na kraju se pokreće događaj $NovaPonuda$, kojim se bilježi adresa korisnika koji je dao novu ponudu i iznos te ponude.

```
function ponudi() public payable {
        require(block.timestamp < vrijemeZavrsetka, "Aukcija je zavrsila.");
        require(msg.value > najvisaPonuda, "Ponuda mora biti veca od trenutne
            najvise ponude.");

        if (najvisaPonuda > 0) {
            najvisiPonuditelj.transfer(najvisaPonuda);
        }

        najvisiPonuditelj = payable(msg.sender);
        najvisaPonuda = msg.value;

        emit NovaPonuda(msg.sender, msg.value);
    }
```

4.1.1.3. Funkcija zavrsiAukciju()

Funkcija zavrsiAukciju() služi za završavanje aukcije. Prvo provjerava je li vrijeme aukcije isteklo pomoću block.timestamp. Ako aukcija još traje, funkcija se neće izvršiti. Zatim provjerava je li aukcija već ranije završena, kako se ista aukcija ne bi mogla završiti više puta.

Ako su uvjeti ispunjeni, varijabla aukcijaZavrsena postavlja se na true. Nakon toga se najveća ponuda šalje vlasniku aukcije pomoću transfer. Na kraju se pokreće događaj AukcijaZavrsena, koji bilježi pobjednika aukcije i iznos pobjedničke ponude.

Ova funkcija koristi require za provjeru uvjeta, a block.timestamp za provjeru vremena na blockchainu.

```
function zavrsiAukciju() public {
        require(block.timestamp >= vrijemeZavrsetka, "Aukcija jos traje.");
        require(!aukcijaZavrsena, "Aukcija je vec zavrsena.");
```

```
        aukcijaZavrsena = true;

        if (najvisaPonuda > 0) {
            vlasnik.transfer(najvisaPonuda);
        }

        emit AukcijaZavrsena(najvisiPonuditelj, najvisaPonuda);
    }
```

4.1.1.4. Funkcija vrijemePreostalo()

Funkcija vrijemePreostalo() služi za prikaz koliko je još vremena ostalo do završetka aukcije. Funkcija je označena kao view, što znači da samo čita podatke iz ugovora i ne mijenja stanje aukcije.

Prvo se provjerava je li trenutno vrijeme veće ili jednako vremenu završetka aukcije. Ako je aukcija već završila, funkcija vraća 0. Ako aukcija još traje, funkcija računa razliku između vrijemeZavrsetka i trenutnog vremena block.timestamp. Rezultat je broj sekundi koji je preostao do kraja aukcije.

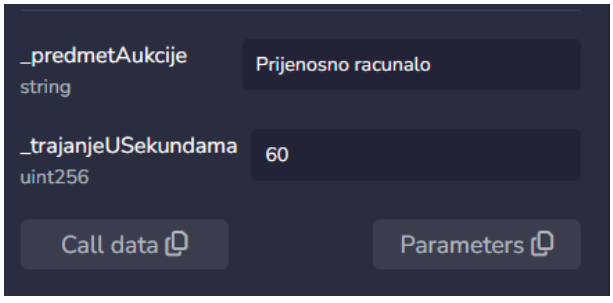
Ovdje se koristi block.timestamp, koji predstavlja vremensku oznaku trenutnog bloka.

```
function vrijemePreostalo() public view returns (uint) {
        if (block.timestamp >= vrijemeZavrsetka) {
            return 0;
        }

        return vrijemeZavrsetka - block.timestamp;
    }
```

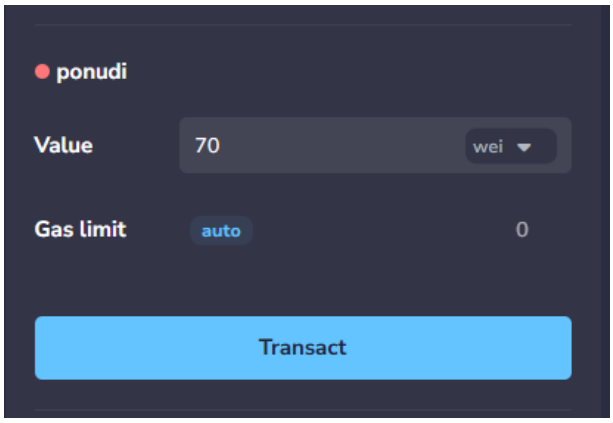
4.1.2. Prikaz rješenja

Slika 2. prikazuje postavljanje aukcije u Remix Ethereum okruženju. U ovom koraku se prije deployanja ugovora unose početni podaci aukcije. U polje $predmetAukcije$ upisan je naziv predmeta koji se prodaje, odnosno „Prijenosno računalo”, a u polje $trajanjeUSekundama$ uneseno je trajanje aukcije od 60 sekundi. Ti se podaci šalju konstruktoru, koji se izvršava samo jednom prilikom kreiranja ugovora.



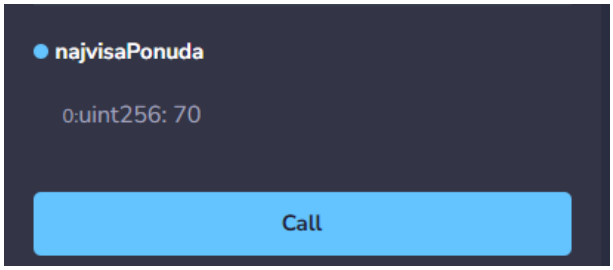
Slika 2: Prkaz postavljanja aukcije

Slika 3. prikazuje funkciju u kojoj korisnik poziva opciju $ponudi()$ i u polje Value unosi iznos ponude. U ovom primjeru unesena je vrijednost 70 wei. Funkcija $ponudi()$ koristi poslani iznos kao novu ponudu i provjerava je li ona veća od trenutne najviše ponude. Vrijednost poslana transakcijom očitava se pomoću msg.value, koji predstavlja broj wei-ja poslanih porukom.



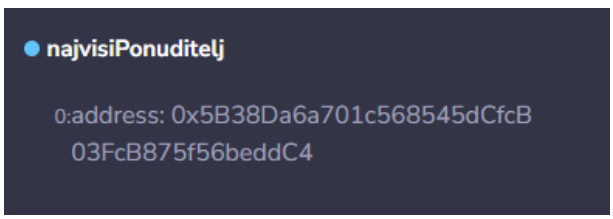
Slika 3: Davanje ponude

Slika 4. prikazuje najvišu ponudu. Nakon uspješnog slanja ponude, klikom na varijablu najvisaPonuda prikazuje se trenutno najveći ponuđeni iznos. U ovom primjeru rezultat je 70, što znači da je najviša ponuda u aukciji 70 wei.



Slika 4: Prkaz najviše ponude

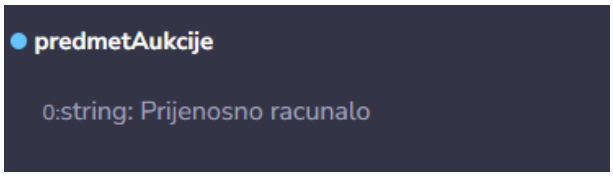
Slika 5. prikazuje najvišeg ponuditelja. Klikom na varijablu $najvisiPonuditelj$ prikazuje se Ethereum adresa korisnika koji je trenutno poslao najveću ponudu. Ta adresa predstavlja trenutnog vodećeg ponuditelja u aukciji.



Slika 5: Prkaz najvišeg ponuditelja

Slika 6. prikazuje naziv predmeta aukcije. Klikom na varijablu $predmetAukcije$ prikazuje se predmet koji je unesen prilikom pokretanja ugovora. U ovom primjeru predmet aukcije je „Prijenosno računalo”.

Slika 7. prikazuje vlasnika aukcije. Varijabla $vlasnik$ prikazuje adresu korisnika koji je



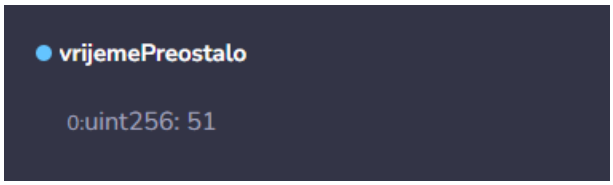
Slika 6: Prkaz naziva predmeta aukcije

deployao pametni ugovor. Ta adresa se sprema pomoću msg.sender, koji označava pošiljatelja poruke, odnosno korisnika koji je pokrenuo određenu radnju u ugovoru.



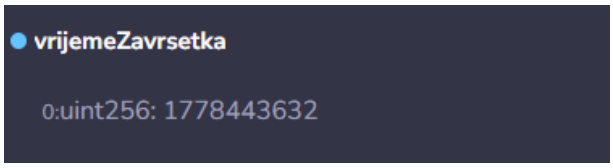
Slika 7: Prkaz vlasnika

Slika 8. prikazuje preostalo vrijeme aukcije. Funkcija $vrijemePreostalo()$ vraća broj sekundi koje su ostale do završetka aukcije. U prikazanom primjeru preostalo je 51 sekunda. Ako je aukcija završila, funkcija vraća vrijednost 0.



Slika 8: Prkaz preostalog vremena

Slika 23. prikazuje vrijeme završetka aukcije. Varijabla $vrijemeZavrsetka$ prikazuje vremensku oznaku do koje aukcija traje. Vrijeme se računa pomoću block.timestamp, koji predstavlja vremensku oznaku trenutnog bloka, i trajanja aukcije koje je uneseno prilikom deployanja ugovora.



Slika 9: Prkaz vremena do završetka aukcije

4.2. Pristup 2: Aukcija s mapiranjem za povrat sredstava

Drugi pristup prikazuje bolju verziju aukcijskog pametnog ugovora. Osnovna ideja ugovora ostaje ista, korisnici daju ponude, a najveća ponuda pobjeđuje nakon završetka aukcije. Razlika je u načinu povrata sredstava prethodnim ponuditeljima.

U ovom pristupu prethodna najviša ponuda ne vraća se odmah starom ponuditelju, nego se sprema u mapiranje povrati. Mapiranje omogućuje spremanje podataka u obliku para ključ-vrijednost, pa se u ovom slučaju adresa ponuditelja povezuje s iznosom koji taj ponuditelj može kasnije povući. Ugovor koristi funkcije za davanje ponude, povlačenje povrata, završetak aukcije i pregled preostalog vremena.

Kod također koristi konstruktor koji se izvršava samo jednom, prilikom kreiranja ugovora. U konstruktoru se postavlja vlasnik aukcije, naziv predmeta i vrijeme završetka aukcije. Za provjeru korisnika, poslanog iznosa i vremena koriste se globalne varijable msg.sender, msg.value i block.timestamp.

U ovom primjeru koda dodani su i funkcijski modifikatori $samoPrijeKraja$ i $samoNakonKraja$. Oni služe za kontrolu kada se pojedina funkcija smije izvršiti. Funkcija ponudi() može se izvršiti samo dok aukcija traje, a funkcija zavrsiAukciju() tek nakon isteka vremena. Ako uvjet nije ispunjen, require prekida izvršavanje funkcije.

Ovaj pristup je pregledniji jer razdvaja logiku davanja ponude od logike povrata sredstava. Zbog toga korisnik koji je izgubio vodeću poziciju može samostalno pozvati funkciju $povuciPovrat()$ i preuzeti iznos koji mu pripada. Programski kod ovog pristupa može se vidjeti u nastavku.

```
pragma solidity >=0.7.0 <0.9.0;

contract AukcijaMapiranjePovrata {
    address payable public vlasnik;
    string public predmetAukcije;
    uint public vrijemeZavrsetka;

    address public najvisiPonuditelj;
    uint public najvisaPonuda;

    bool public aukcijaZavrsena;

    mapping(address => uint) public povrati;

    event NovaPonuda(address ponuditelj, uint iznos);
    event PovratSredstava(address ponuditelj, uint iznos);
    event AukcijaZavrsena(address pobjednik, uint iznos);

    modifier samoPrijeKraja() {
        require(block.timestamp < vrijemeZavrsetka, "Aukcija je zavrsila.");
        _;
    }

    modifier samoNakonKraja() {
        require(block.timestamp >= vrijemeZavrsetka, "Aukcija jos traje.");
        _;
    }

    constructor(string memory _predmetAukcije, uint _trajanjeUSekundama) {
        vlasnik = payable(msg.sender);
        predmetAukcije = _predmetAukcije;
```

```
        vrijemeZavrsetka = block.timestamp + _trajanjeUSekundama;
    }

    function ponudi() public payable samoPrijeKraja {
        require(msg.value > najvisaPonuda, "Ponuda␣mora␣biti␣veca␣od␣trenutne␣
            najvise␣ponude.");

        if (najvisaPonuda > 0) {
            povrati[najvisiPonuditelj] += najvisaPonuda;
        }

        najvisiPonuditelj = msg.sender;
        najvisaPonuda = msg.value;

        emit NovaPonuda(msg.sender, msg.value);
    }

    function povuciPovrat() public {
        uint iznos = povrati[msg.sender];

        require(iznos > 0, "Nema␣sredstava␣za␣povrat.");

        povrati[msg.sender] = 0;
        payable(msg.sender).transfer(iznos);

        emit PovratSredstava(msg.sender, iznos);
    }

    function zavrsiAukciju() public samoNakonKraja {
        require(!aukcijaZavrsena, "Aukcija␣je␣vec␣zavrsena.");

        aukcijaZavrsena = true;

        if (najvisaPonuda > 0) {
            vlasnik.transfer(najvisaPonuda);
        }

        emit AukcijaZavrsena(najvisiPonuditelj, najvisaPonuda);
    }

    function vrijemePreostalo() public view returns (uint) {
        if (block.timestamp >= vrijemeZavrsetka) {
            return 0;
        }

        return vrijemeZavrsetka - block.timestamp;
    }

    function mojPovrat() public view returns (uint) {
        return povrati[msg.sender];
    }
}
```

4.2.1. Objašnjenje programskog rješenja

4.2.1.1. Struktura pametnog ugovora

Ovaj dio koda je ugovor $AukcijaMapiranjePovrata$. Na početku se definiraju varijable koje spremaju podatke o aukciji: vlasnika, naziv predmeta, vrijeme završetka, trenutnog najvišeg ponuditelja, najveću ponudu i informaciju je li aukcija završena.

Važan dio je mapping(address => uint) public povrati, koji povezuje adresu ponuditelja s iznosom koji može kasnije povući ako njegova ponuda bude nadmašena. Mapiranje radi kao zapis u obliku ključ-vrijednost, gdje je ključ adresa korisnika, a vrijednost iznos za povrat.

Događaji $NovaPonuda$, $PovratSredstava$ i $AukcijaZavrsena$ služe za bilježenje važnih radnji u aukciji, kao što su nova ponuda, povrat sredstava i završetak aukcije.

Modifikatori $samoPrijeKraja$ i $samoNakonKraja$ koriste se za kontrolu vremena. Prvi dopušta izvršavanje funkcije samo dok aukcija traje, a drugi tek nakon završetka aukcije. U njima se koristi require, koji prekida izvođenje ako uvjet nije zadovoljen.

Konstruktor se izvršava samo jednom, prilikom kreiranja ugovora. U njemu se postavlja vlasnik aukcije pomoću msg.sender, sprema se naziv predmeta aukcije i računa vrijeme završetka pomoću block.timestamp i unesenog trajanja aukcije.

```
pragma solidity >=0.7.0 <0.9.0;

contract AukcijaMapiranjePovrata {
    address payable public vlasnik;
    string public predmetAukcije;
    uint public vrijemeZavrsetka;

    address public najvisiPonuditelj;
    uint public najvisaPonuda;

    bool public aukcijaZavrsena;

    mapping(address => uint) public povrati;

    event NovaPonuda(address ponuditelj, uint iznos);
    event PovratSredstava(address ponuditelj, uint iznos);
    event AukcijaZavrsena(address pobjednik, uint iznos);

    modifier samoPrijeKraja() {
        require(block.timestamp < vrijemeZavrsetka, "Aukcija␣je␣zavrsila.");
        _;
    }

    modifier samoNakonKraja() {
        require(block.timestamp >= vrijemeZavrsetka, "Aukcija␣jos␣traje.");
        _;
    }

    constructor(string memory _predmetAukcije, uint _trajanjeUSekundama) {
```

```
        vlasnik = payable(msg.sender);
        predmetAukcije = _predmetAukcije;
        vrijemeZavrsetka = block.timestamp + _trajanjeUSekundama;
    }
```

4.2.1.2. Funkcija ponudi()

Funkcija ponudi() služi za davanje nove ponude u aukciji. Označena je kao payable, što znači da korisnik uz poziv funkcije može poslati određeni iznos Ethera. Iznos koji korisnik pošalje provjerava se pomoću msg.value, a funkcija se može izvršiti samo dok aukcija traje zbog modifikatora samoPrijeKraja. Prvo se provjerava je li nova ponuda veća od trenutne najviše ponude. Ako nije veća, funkcija se prekida pomoću require. Ako već postoji prethodna najviša ponuda, taj se iznos ne vraća odmah korisniku, nego se sprema u mapiranje povrati pod adresom prethodnog najvišeg ponuditelja.

Nakon toga se nova adresa ponuditelja sprema u najvisiPonuditelj, a poslani iznos u najvisaPonuda. Na kraju se pokreće događaj NovaPonuda, koji bilježi tko je poslao ponudu i koliki je iznos ponude.

```
    function ponudi() public payable samoPrijeKraja {
        require(msg.value > najvisaPonuda, "Ponuda␣mora␣biti␣veca␣od␣trenutne␣
            najvise␣ponude.");

        if (najvisaPonuda > 0) {
            povrati[najvisiPonuditelj] += najvisaPonuda;
        }

        najvisiPonuditelj = msg.sender;
        najvisaPonuda = msg.value;

        emit NovaPonuda(msg.sender, msg.value);
    }
```

4.2.1.3. Funkcija povuciPovrat()

Funkcija povuciPovrat() služi za povrat sredstava korisniku čija je ponuda bila nadmašena. Prvo se iz mapiranja povrati dohvaća iznos koji pripada korisniku koji je pozvao funkciju. Za to se koristi msg.sender, koji označava trenutnog pozivatelja funkcije.

Nakon toga se pomoću require provjerava ima li korisnik uopće sredstva za povrat. Ako je iznos jednak nuli, funkcija se prekida. Ako iznos postoji, vrijednost u mapiranju se prvo postavlja na 0, a zatim se korisniku šalju sredstva pomoću transfer.

Na kraju se pokreće događaj PovratSredstava, koji bilježi adresu korisnika i iznos koji mu je vraćen. Ova funkcija je važna jer omogućuje da korisnici sami povuku svoj povrat nakon što njihova ponuda više nije najviša.

```
function povuciPovrat() public {
        uint iznos = povrati[msg.sender];
```

```
        require(iznos > 0, "Nema sredstava za povrat.");

        povrati[msg.sender] = 0;
        payable(msg.sender).transfer(iznos);

        emit PovratSredstava(msg.sender, iznos);
    }
```

4.2.1.4. Funkcija zavrsiAukciju()

Funkcija $zavrsiAukciju()$ služi za završavanje aukcije u drugom pristupu. Zbog modifikatora $samoNakonKraja$, funkcija se može izvršiti tek nakon što istekne vrijeme aukcije. Modifikatori se koriste za dodavanje uvjeta funkcijama, a ako uvjet nije zadovoljen, funkcija se ne izvršava.

Nakon toga se pomoću require provjerava je li aukcija već završena. Ako nije, varijabla $aukcijaZavrsena$ postavlja se na true. Ako postoji najviša ponuda, njezin iznos se šalje vlasniku aukcije pomoću transfer. Na kraju se pokreće događaj $AukcijaZavrsena$, koji bilježi adresu pobjednika i iznos pobjedničke ponude.

```
function zavrsiAukciju() public samoNakonKraja {
        require(!aukcijaZavrsena, "Aukcija je vec zavrsena.");

        aukcijaZavrsena = true;

        if (najvisaPonuda > 0) {
            vlasnik.transfer(najvisaPonuda);
        }

        emit AukcijaZavrsena(najvisiPonuditelj, najvisaPonuda);
    }
```

4.2.1.5. Funkcija vrijemePreostalo() i mojPovrat()

Funkcija $vrijemePreostalo()$ služi za prikaz koliko je još sekundi ostalo do završetka aukcije. Funkcija je označena kao view, što znači da samo čita podatke iz ugovora i ne mijenja stanje. Ako je aukcija već završila, vraća vrijednost 0. Ako još traje, računa razliku između vrijemeZavrsetka i trenutnog vremena block.timestamp. block.timestamp predstavlja vremensku oznaku trenutnog bloka.

Funkcija $mojPovrat()$ služi da korisnik provjeri koliko sredstava može povući natrag. Ona iz mapiranja povrati dohvaća iznos koji je spremljen za adresu korisnika koji je pozvao funkciju, odnosno za msg.sender. Mapiranje povezuje Ethereum adresu s pripadajućom vrijednosti, pa se ovdje koristi za prikaz povrata pojedinom ponuditelju.

```
    function vrijemePreostalo() public view returns (uint) {
        if (block.timestamp >= vrijemeZavrsetka) {
            return 0;
```

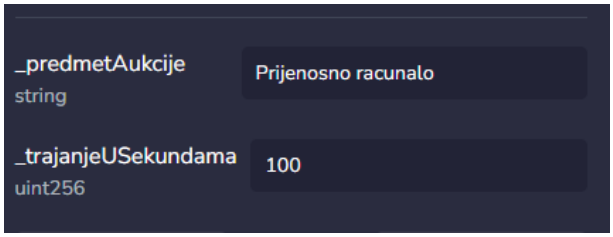
```
        }

        return vrijemeZavrsetka - block.timestamp;
    }

    function mojPovrat() public view returns (uint) {
        return povrati[msg.sender];
    }
}
```

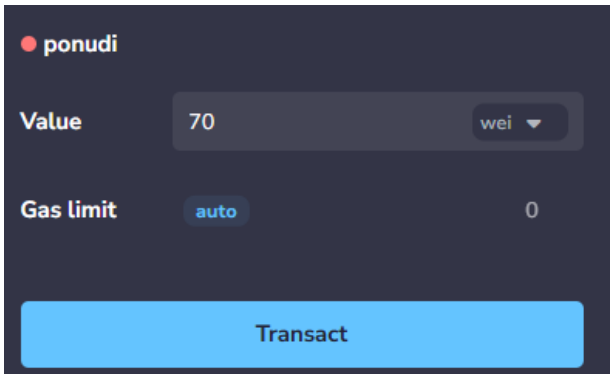
4.2.2. Prikaz rješenja

Slika 10. prikazuje postavljanje druge aukcije. Na slici se vidi unos početnih podataka prije deployanja pametnog ugovora. U polje $predmetAukcije$ upisan je naziv predmeta „Prijenosno računalo", a u polje $trajanjeUSekundama$ unesena je vrijednost 100, što znači da aukcija traje 100 sekundi. Ti se podaci šalju konstruktoru, koji se izvršava jednom prilikom kreiranja ugovora i postavlja početno stanje aukcije.



Slika 10: Postavljanje druge aukcije

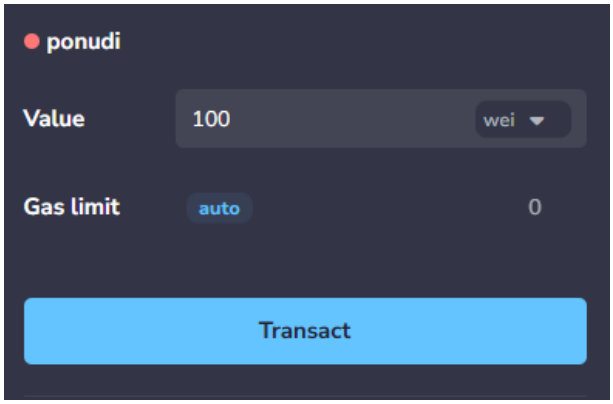
Slika 11. prikazuje predaju prve ponude. U ovom koraku korisnik poziva funkciju $ponudi()$ i u polje Value unosi iznos 70 wei. Time korisnik šalje prvu ponudu u aukciji. Budući da prije toga nije bilo veće ponude, ugovor prihvaća ovu ponudu i sprema korisnika kao trenutnog najvišeg ponuditelja, a iznos 70 kao trenutnu najvišu ponudu. msg.value predstavlja iznos wei-ja poslanih transakcijom.



Slika 11: Predaja prve ponude

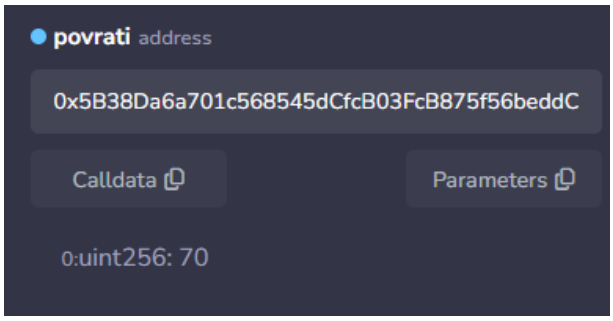
Slika 12. prikazuje predaju druge ponude. Na slici se vidi da drugi korisnik unosi novu ponudu od 100 wei. Ta ponuda je veća od prethodne ponude od 70 wei, pa ugovor prihvaća novu ponudu. Novi korisnik postaje $najvisiPonuditelj$, a $najvisaPonuda$ se mijenja na 100

wei. Kod ovog drugog pristupa prethodna ponuda od 70 wei ne vraća se odmah, nego se sprema u mapiranje povrati, kako bi je prethodni ponuditelj mogao kasnije sam povući. Mapiranje povezuje adresu korisnika s pripadajućom vrijednosti.



Slika 12: Predaja druge ponude

Slika 13. prikazuje funkciju povrati. Na slici je u polje funkcije povrati unesena adresa korisnika kojem je prethodna ponuda nadmašena. Funkcija vraća vrijednost 70, što znači da taj korisnik ima spremljen povrat od 70 wei. Ova funkcija radi preko mapiranja povrati, gdje se adresa korisnika povezuje s iznosom koji može kasnije povući.



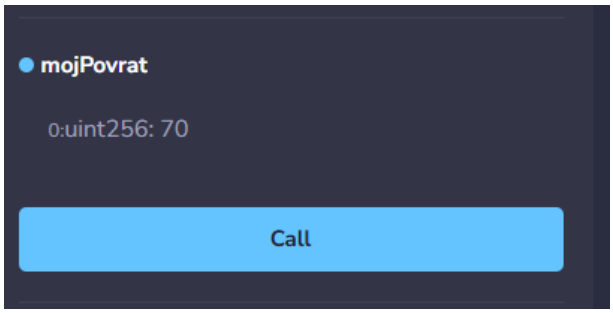
Slika 13: Funcija povratAdresa

Slika 14. prikazuje funkciju $mojPovrat$. Funkcija $mojPovrat()$ prikazuje koliko trenutno odabrani korisnik može povući natrag. Rezultat je 70, što znači da taj korisnik ima pravo na povrat od 70 wei. Razlika između $povrati$ i $mojPovrat$ je u tome što se kod povrati ručno upisuje adresa, a kod mojPovrat ugovor sam provjerava adresu korisnika koji je trenutno odabran pomoću msg.sender.

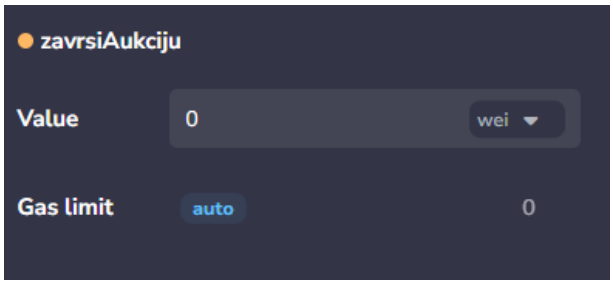
Slika 16. prikazuje pozivanje funkcije $zavrsiAukciju$. Na slici je prikazano da se funkcija $zavrsiAukciju()$ poziva s vrijednosti 0 wei. To je ispravno jer se kod završavanja aukcije ne šalje nova ponuda, nego se samo zaključuje aukcija nakon isteka vremena.

Slika 16. također prikazuje funkciju $zavrsiAukciju$. Ovdje se ponovno vidi pozivanje iste funkcije s Value = 0 wei. Funkcija se može uspješno izvršiti tek kada je prošlo vrijeme trajanja aukcije. Ako aukcija još traje, ugovor neće dopustiti završetak zbog uvjeta u modifikatoru $samoNakonKraja$.

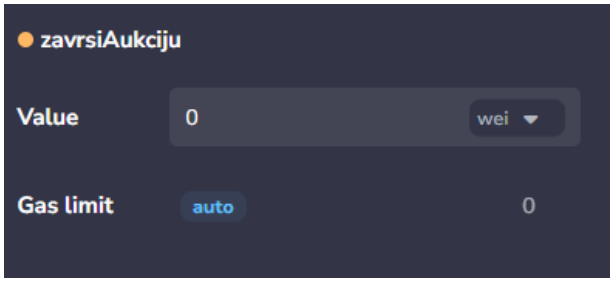
Slika 17. prikazuje funkciju za prikaz vlasnika aukcije. Klikom na varijablu vlasnik pri-



Slika 14: Funkcija mojPovrat



Slika 15: Funkcija zavrsiAukciju



Slika 16: Funkcija zavrsiAukciju

kazuje se Ethereum adresa korisnika koji je deployao ugovor. Ta adresa predstavlja vlasnika aukcije, odnosno osobu kojoj se nakon završetka aukcije šalje najviša ponuda. Adresa vlasnika postavlja se u konstruktoru pomoću msg.sender.



Slika 17: Funkcija za prikaz vlasnika aukcije

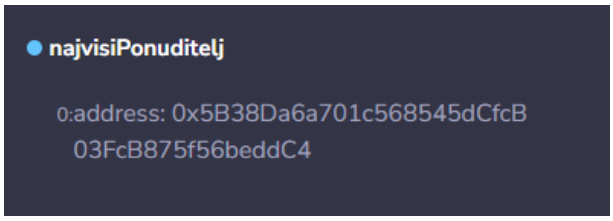
Slika 18. prikazuje funkciju za prikaz najviše ponude. Klikom na varijablu $najvisaPonuda$ prikazuje se trenutno najveći iznos koji je ponuđen u aukciji. U ovom primjeru rezultat je 100, što znači da je najviša ponuda nakon druge predaje ponude 100 wei. Budući da je varijabla označena kao public, njezina se vrijednost može direktno prikazati u Remixu.

Slika 19. prikazuje funkciju za prikaz najvišeg ponuditelja. Klikom na varijablu $najvisiPonuditelj$



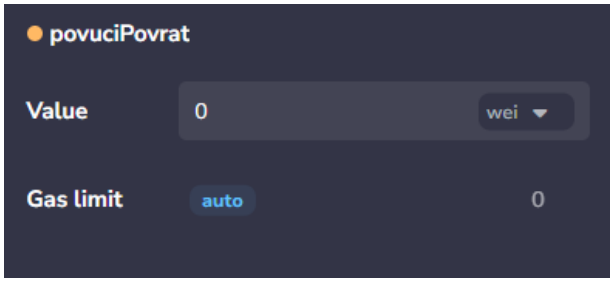
Slika 18: Funkcija za prikaz najviše pondude

prikazuje se Ethereum adresa korisnika koji trenutno ima najveću ponudu. U ovom slučaju prikazana adresa pripada korisniku koji je dao ponudu od 100 wei, pa je on trenutno vodeći ponuditelj u aukciji.



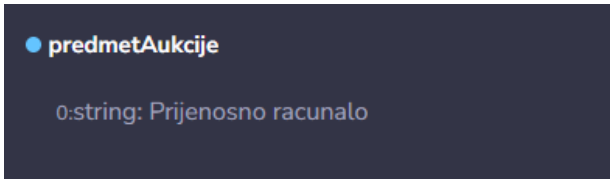
Slika 19: Funkcija za prikaz najvišeg ponuditelja

Slika 20. prikazuje funkciju $povuciPovrat$. Ova funkcija služi za povlačenje povrata sredstava korisniku čija je ponuda nadmašena. Na slici je vrijednost Value = 0 wei, što je ispravno jer se kod ove funkcije ne šalje nova ponuda, nego korisnik samo povlači ranije spremljeni iznos. Ako korisnik ima povrat, funkcija mu šalje sredstva, a nakon toga se njegov povrat postavlja na 0. Funkcija koristi msg.sender kako bi znala koji korisnik traži povrat.



Slika 20: Funkcija za prikaz povrata

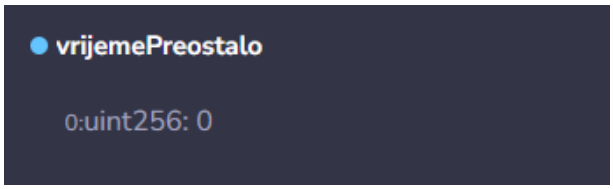
Slika 21. prikazuje funkciju za prikaz predmeta aukcije. Pritiskom na varijablu $predmetAukcije$ prikazuje se naziv predmeta koji je unesen prilikom pokretanja ugovora. U ovom primjeru predmet aukcije je „Prijenosno računalo". Taj podatak je spremljen u konstruktoru prilikom deployanja pametnog ugovora.



Slika 21: Funkcija za predmeta aukcije

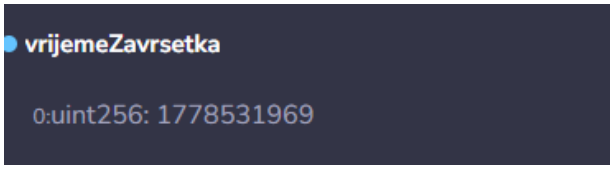
Slika 22 prikazuje funkciju za prikaz preostalog vremena aukcije. Funkcija $vrijemePreostalo()$

prikazuje koliko je sekundi ostalo do završetka aukcije. Na slici je prikazana vrijednost 0, što znači da je vrijeme aukcije isteklo. Nakon toga više nije moguće dati novu ponudu, nego se aukcija može završiti funkcijom $zavrsiAukciju()$.



Slika 22: Funkcija za prikaz preostalog vremena aukcije

Slika 23. prikazuje vrijeme završetka aukcije. Klikom na varijablu $vrijemeZavrsetka$ prikazuje se vremenska oznaka trenutka kada aukcija završava. Ta vrijednost se računa prilikom deployanja ugovora tako da se trenutnom vremenu bloka doda trajanje aukcije koje je korisnik unio. Za to se koristi block.timestamp.



Slika 23: Funkcija za prikaz preostalog vremena završetka aukcije

5. Zaključak

U teorijskom dijelu rada objašnjeni su osnovni pojmovi blockchaina, način rada blockchain mreže, konsenzusni mehanizmi, kriptografija i pametni ugovori. Također je prikazana uloga Ethereuma kao platforme koja omogućuje razvoj pametnih ugovora i decentraliziranih aplikacija. Ethereum je važan jer proširuje primjenu blockchaina izvan same razmjene vrijednosti i omogućuje izvršavanje programskog koda unutar blockchain okruženja. [3]

Praktični dio rada temelji se na izradi aukcijskog pametnog ugovora u Solidityju. Izrađena su dva pristupa istom problemu. Prvi pristup, *AukcijaDirektanPovrat*, prikazuje jednostavniju verziju aukcije u kojoj se prethodnom najvišem ponuditelju sredstva odmah vraćaju kada se pojavi nova veća ponuda. Ovaj način je jednostavan za razumijevanje i prikladan za osnovni prikaz rada aukcije.

Drugi pristup, *AukcijaMapiranjePovrata*, koristi mapiranje za spremanje iznosa koji korisnici mogu naknadno povući. Time se povrat sredstava odvaja od same funkcije davanja ponude, što rješenje čini preglednijim i bolje organiziranim. U ovom pristupu korišteni su važni elementi Solidityja kao što su funkcije, konstruktor, mapiranje, globalne varijable, modifikatori i Ether jedinice.

Testiranjem u Remix VM okruženju prikazano je kako se ugovor deploya, kako korisnici daju ponude, kako se prati najviša ponuda, kako se provjerava najviši ponuditelj te kako se završava aukcija. Kod drugog pristupa je prikazano kako se korisniku čija je ponuda nadmašena sprema iznos za povrat i kako ga može samostalno povući.

Na temelju izrađenog rješenja može se zaključiti da pametni ugovori mogu automatizirati jednostavne poslovne procese poput aukcije. Pravila aukcije unaprijed su definirana u kodu, a njihovo izvršavanje ne ovisi o posredniku, nego o uvjetima zapisanima u pametnom ugovoru. Na taj se način povećava transparentnost, smanjuje mogućnost pogreške i pokazuje kako se blockchain može koristiti u stvarnim primjerima digitalnog poslovanja.

U projeku se pokazuje da se klasičan proces aukcije može uspješno prenijeti u blockchain okruženje. Prvi pristup prikazuje osnovnu logiku aukcije, dok drugi pristup predstavlja naprednije i praktičnije rješenje zbog korištenja mapiranja povrata. Time je prikazano kako Solidity i Ethereum mogu služiti za razvoj pametnih ugovora koji automatiziraju poslovne procese i predstavljaju važan dio suvremenog digitalnog poslovanja.

Popis literature

[1] N. Vrček, *Upravljanje digitalnim inovacijama*, Nastavni materijali, Fakultet organizacije i informatike, Varaždin, 2024.

[2] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, https://www.bitcoin.org/bitcoin.pdf, 2008.

[3] V. Buterin, *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*, https://ethereum.org/en/whitepaper/, 2014.

[4] T. Huynh-The i dr., „Blockchain for the Metaverse: A Review," *Future Generation Computer Systems*, sv. 143, str. 401–419, 2023. DOI: 10.1016/j.future.2023.02.008.

[5] G. Wood, „Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Ethereum Project Yellow Paper*, 2014. adresa: https://ethereum.github.io/yellowpaper/paper.pdf.

Popis slika

1. Prkaz use case dijagrama (Izvor: vlastita izrada) 9
2. Prkaz postavljanja aukcije 14
3. Davanje ponude 15
4. Prkaz najviše ponude 15
5. Prkaz najvišeg ponuditelja 15
6. Prkaz naziva predmeta aukcije 16
7. Prkaz vlasnika 16
8. Prkaz preostalog vremena 16
9. Prkaz vremena do završetka aukcije 16
10. Postavljanje druge aukcije 22
11. Predaja prve ponude 22
12. Predaja druge ponude 23
13. Funcija povratAdresa 23
14. Funkcija mojPovrat 24
15. Funkcija zavrsiAukciju 24
16. Funkcija zavrsiAukciju 24
17. Funkcija za prikaz vlasnika aukcije 24
18. Funkcija za prikaz najviše pondude 25
19. Funkcija za prikaz najvišeg ponuditelja 25
20. Funkcija za prikaz povrata 25
21. Funkcija za predmeta aukcije 25
22. Funkcija za prikaz preostalog vremena aukcije 26
23. Funkcija za prikaz preostalog vremena završetka aukcije 26